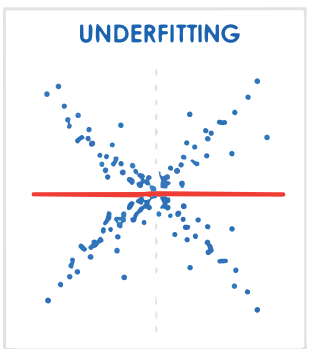
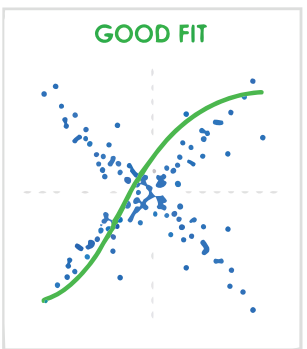
Understanding Underfitting and Overfitting in Machine Learning

Machine learning models must balance the concepts of underfitting and overfitting.



Model too simple, high error on training and test data



Just right, low error on training and test data

Model too complex, low error on training, high error on test data

In machine learning, a model's goal is not just to fit the given data, but to generalise well to unseen data. Two common failure modes prevent this:

- Underfitting – the model is too simple to capture the true pattern.
- Overfitting – the model is too complex and memorises the training data.

I will demonstrate both concepts using simple, executable Python code, and a dataset generated from a known mathematical formula. This makes it easy to compare predicted values with true values.

Dataset with known ground truth

To clearly understand model behaviour, we generate data using a known formula:

$$y = x^2 + 2x + 1$$

This quadratic relationship allows us to check:

- How well a model fits training data
- How it behaves on unseen values

```
import numpy as np
X = np.array([[1], [2], [3], [4], [5], [6], [7], [8], [9],
[10]])
y = np.array([x[0]**2 + 2*x[0] + 1 for x in X])
```

Underfitting example: Linear regression

Linear regression assumes a straight-line relationship between input and output. Since our data is quadratic, this assumption is incorrect. Here's the code:

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

# Train
model.fit(X, y)

# Predict
print("Underfitting (Linear Regression)")
print("x | predicted | true")
print("--+-----------+------")

print("Seen (training range)")
for v in [1, 3, 5, 7, 9]:
    y_pred = model.predict([[v]])[0]
    y_true = v*v + 2*v + 1
    print(f"{v:2d} | {y_pred:9.2f} | {y_true:4d}")

print("\nUnseen (outside training range)")
```